

calculate_postTARE_ts_aws_peak_demand_20April2026

April 20, 2026

Post-TARE Model Run: Timeseries Peak Load Analysis (AWS / BuildStockQuery)

Author: Jordan M. Joseph, PhD — Carnegie Mellon University

Queries the AWS-hosted ResStock EUSS 2022.1.1 timeseries database via BuildStockQuery to compute **county-level peak load changes** under two adoption scenarios: - **(a) 100% adoption counterfactual** — all filtered building IDs adopt (upper bound) - **(b) Economically-constrained adoption** — only Tier 1 + Tier 2 adopters (TARE output)

Prerequisite: Steps 0–0c must be run first (TARE data loaded, MP selected).

Dataset scope: ResStock 2022.1.1 (EUSS, AMY2018). ResStock 2025.1 is future work.

Primary test case: Allegheny County, PA (FIPS 42003) — validate here before scaling.

See methodology notes in `peak_load_methodology.md`.

Step 0: Imports and Configuration

```
[1]: import os
import time
from typing import Any, Final

import pandas as pd
import numpy as np
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

from config import PROJECT_ROOT
from cmu_tare_model.constants import (
    ALLOWED_HOUSING_TYPES,
    VALID_MENU_MPS,
    VERBOSE,
    REMDB_COST_SCENARIO_KEYS,
    RCM_MODELS,
    PRIVATE_DISCOUNT_RATE_SHORT_KEYS,
    BLDG_ID_COL,
    TIMESTAMP_COL,
```

```

ELEC_TOTAL_COL,
BSQ_ELEC_COL,
METADATA_TABLE,
COUNTY_COL,
STATE_COL,
WEIGHT_COL,
TEST_FIPS,
TEST_GISJOIN,
)

from cmu_tare_model.utils.column_names import (
    create_npv_col,
    create_capital_col,
)

from cmu_tare_model.utils.load_exported_results_to_df import (
    ↪load_measure_package_data

from cmu_tare_model.adoption_kpis.kpi_functions import (
    mp_to_upgrade,
    load_euss_baseline,
    load_euss_upgrade,
    calculate_price_ratios,
    compute_thermal_cop_by_state,
    compute_spark_gap_metrics,
    compute_scenario_demand,
    aggregate_demand_by_state,
    FUEL_PRICES_PATH,
    SHAPEFILE_PATH,
    HEATING_FUEL_COLS,
    HP_BACKUP_ELEC_COL,
    HP_FANS_PUMPS_COL,
)
from cmu_tare_model.adoption_kpis.peak_load_functions import (
    gisjoin_to_fips,
    find_adoption_column,
    extract_adopter_ids,
    compute_county_scenario_profile,
)
from cmu_tare_model.adoption_kpis.visualize_geospatial_data import (
    prepare_state_geodataframe,
    create_choropleth_map,
)

pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 60)

```

```
print(" Imports loaded")
```

Project root directory: c:\users\jorda\Desktop\projects\cmu-tare-model
Imports loaded

Step 0b: Measure Package Selection

```
[2]: # =====  
# STEP 0b: Measure Package Selection (hardcoded for reproducibility)  
# =====  
# Previously used input() - replaced with constants for non-interactive runs.  
# Change selected_mps to run different measure packages.  
  
SELECTABLE_MPS = [mp for mp in VALID_MENU_MPS if mp != 0]  
  
# selected_mps: list[int] = [3] # MP3 = ducted ASHP (primary analysis)  
selected_mps: list[int] = [3, 4]  
batch_mode: bool = False  
  
print(f"Selected measure packages: {selected_mps}")  
print(f"Available: {SELECTABLE_MPS}")
```

Selected measure packages: [3, 4]
Available: [3, 4]

Step 0c: Load TARE Model Data (Measure Packages 3, 4)

Load pre-computed TARE model outputs for the selected measure packages. Required for Step 4d (Private NPV extraction).

If the top-section data loading cells have already been run, this will reuse DATAFRAMES_BY_MP.

```
[3]: # =====  
# STEP 0c: LOAD TARE MODEL DATA (for NPV extraction)  
# =====  
# Check if DATAFRAMES_BY_MP was already loaded by the top-section cells.  
# If not, prompt for the output folder and load for selected MPs.  
  
try:  
    _ = DATAFRAMES_BY_MP  
    print(f"DATAFRAMES_BY_MP already loaded: {list(DATAFRAMES_BY_MP.keys())}")  
except NameError:  
    print("DATAFRAMES_BY_MP not found - loading TARE model outputs...")  
  
    # Check if output_folder_path is already defined (from top section)  
    try:  
        _ = output_folder_path
```

```

    print(f" Using existing output_folder_path: {output_folder_path}")
except NameError:
    output_folder_path = os.path.join(PROJECT_ROOT, "cmu_tare_model",
↪ "output_results")
    location_id = input("Enter location ID (e.g., 'National' or 'PA'): ").
↪ strip()
    model_run_date_time = input("Enter model run timestamp
↪ (YYYY-MM-DD_HH-MM): ").strip()
    print(f" output_folder_path: {output_folder_path}")
    print(f" location_id: {location_id}")
    print(f" model_run_date_time: {model_run_date_time}")

DATAFRAMES_BY_MP = {}
for mp in selected_mps:
    DATAFRAMES_BY_MP[mp] = load_measure_package_data(
        mp, output_folder_path, location_id, model_run_date_time
    )

print(f"\n Loaded TARE data for MPs: {list(DATAFRAMES_BY_MP.keys())}")

```

```

DATAFRAMES_BY_MP not found - loading TARE model outputs...
  output_folder_path: c:\users\jorda\desktop\projects\cmu-tare-
model\cmu_tare_model\output_results
  location_id: National
  model_run_date_time: 2026-04-10_00-05
Loading MP3 data...
  fixed_base:
MP3 loading complete!

Loading MP4 data...
  fixed_base:
MP4 loading complete!

```

```
Loaded TARE data for MPs: [3, 4]
```

Step 1: BuildStockQuery Initialization

Initializes BuildStockQuery (NREL SWR-23-58) to connect to the OEDI data lake via AWS Athena. BSQ handles all Athena infrastructure (workgroup, Glue catalog, partition registration, query execution) internally.

0.0.1 Prerequisites

1. `pip install git+https://github.com/NREL/buildstock-query.git`
2. `aws configure` with valid credentials (region: `us-west-2`)
3. IAM policies: `AmazonAthenaFullAccess`, `AmazonS3ReadOnlyAccess`

0.0.2 BSQ Parameters (EUSS 2022.1.1)

Parameter	Value
workgroup	resstock-euss
db_name	euss-oedi
table_name	resstock_amy2018_release_1_1
db_schema	resstock_oedi
buildstock_type	resstock

0.0.3 Weight note

All 548,916 EUSS 2022.1.1 buildings have **uniform weight = 242.131013**. BSQ applies this automatically via $\text{SUM}(\text{enduse} \times \text{weight})$ in generated SQL. Previous hardcoded of 240.0 has been removed (0.9% error).

0.0.4 Known issue

`split_enduses=True` triggers a Pydantic `ValidationError` in BSQ's batch query path. Use `split_enduses=False` with single enduse (works correctly).

```
[4]: # =====
# STEP 1: BuildStockQuery Initialization
# =====

# ===== 1. Import BuildStockQuery =====
from buildstock_query import BuildStockQuery # type: ignore[import-untyped]
from buildstock_query.schema.query_params import TSQuery
print(f" BuildStockQuery imported")

# ===== 2. Verify AWS credentials =====
import boto3
from botocore.exceptions import NoCredentialsError, ClientError

session = boto3.session.Session()
aws_region: str | None = session.region_name

try:
    sts = session.client("sts")
    aws_identity: dict = sts.get_caller_identity()
    print(f"""
        AWS credentials valid
        Account : {aws_identity['Account']}
        ARN      : {aws_identity['Arn']}
        Region   : {aws_region}
        """)
except NoCredentialsError:
```

```

        raise RuntimeError(
            "AWS credentials not found. Run `aws configure` or set "
            "AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY environment variables."
        )
    except ClientError as e:
        raise RuntimeError(f"AWS STS call failed: {e}")

# ===== 3. Initialize BuildStockQuery =====
my_run = BuildStockQuery(
    workgroup='resstock-euss',
    db_name='euss-oedi',
    table_name='resstock_amy2018_release_1_1',
    db_schema='resstock_oedi',
    buildstock_type='resstock',
    skip_reports=True,
)

print(f"""
    BuildStockQuery initialized: {type(my_run).__name__}

    Metadata table : {my_run.bs_table}
    Timeseries table: {my_run.ts_table}
    Building ID col : {my_run.building_id_column_name}

    Step 1 COMPLETE
    """)

```

```

INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials

```

```

    BuildStockQuery imported

```

```

INFO:buildstock_query.query_core:Loading resstock_amy2018_release_1_1 ...
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials

```

```

    AWS credentials valid
    Account : 069765036196
    ARN      : arn:aws:iam::069765036196:user/jordanjo@alumni.cmu.edu
    Region   : us-west-2

```

```

INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials
INFO:botocore.credentials:Found credentials in shared credentials file:
~/.aws/credentials

```

BuildStockQuery initialized: BuildStockQuery

Metadata table : SELECT resstock_amy2018_release_1_1_metadata.applicability,
resstock_amy2018_release_1_1_metadata."in.sqft",
resstock_amy2018_release_1_1_metadata.weight,
resstock_amy2018_release_1_1_metadata."in.ahs_region",
resstock_amy2018_release_1_1_metadata."in.ashrae_iecc_climate_zone_2004", ressto
ck_amy2018_release_1_1_metadata."in.ashrae_iecc_climate_zone_2004_2_a_split",
resstock_amy2018_release_1_1_metadata."in.bathroom_spot_vent_hour",
resstock_amy2018_release_1_1_metadata."in.bedrooms",
resstock_amy2018_release_1_1_metadata."in.building_america_climate_zone",
resstock_amy2018_release_1_1_metadata."in.cec_climate_zone",
resstock_amy2018_release_1_1_metadata."in.ceiling_fan",
resstock_amy2018_release_1_1_metadata."in.census_division",
resstock_amy2018_release_1_1_metadata."in.census_division_rec",
resstock_amy2018_release_1_1_metadata."in.census_region",
resstock_amy2018_release_1_1_metadata."in.city",
resstock_amy2018_release_1_1_metadata."in.clothes_dryer",
resstock_amy2018_release_1_1_metadata."in.clothes_washer",
resstock_amy2018_release_1_1_metadata."in.clothes_washer_presence",
resstock_amy2018_release_1_1_metadata."in.cooking_range",
resstock_amy2018_release_1_1_metadata."in.cooling_setpoint",
resstock_amy2018_release_1_1_metadata."in.cooling_setpoint_has_offset",
resstock_amy2018_release_1_1_metadata."in.cooling_setpoint_offset_magnitude",
resstock_amy2018_release_1_1_metadata."in.cooling_setpoint_offset_period",
resstock_amy2018_release_1_1_metadata."in.corridor",
resstock_amy2018_release_1_1_metadata."in.county",
resstock_amy2018_release_1_1_metadata."in.county_and_puma",
resstock_amy2018_release_1_1_metadata."in.dehumidifier",
resstock_amy2018_release_1_1_metadata."in.dishwasher",
resstock_amy2018_release_1_1_metadata."in.door_area",
resstock_amy2018_release_1_1_metadata."in.doors",
resstock_amy2018_release_1_1_metadata."in.ducts",
resstock_amy2018_release_1_1_metadata."in.eaves",
resstock_amy2018_release_1_1_metadata."in.electric_vehicle",
resstock_amy2018_release_1_1_metadata."in.emissions_electricity_folders",
resstock_amy2018_release_1_1_metadata."in.emissions_electricity_units", resstock
_amy2018_release_1_1_metadata."in.emissions_electricity_values_or_filepaths",
resstock_amy2018_release_1_1_metadata."in.emissions_fossil_fuel_units",
resstock_amy2018_release_1_1_metadata."in.emissions_fuel_oil_values",
resstock_amy2018_release_1_1_metadata."in.emissions_natural_gas_values",
resstock_amy2018_release_1_1_metadata."in.emissions_propane_values",
resstock_amy2018_release_1_1_metadata."in.emissions_scenario_names",
resstock_amy2018_release_1_1_metadata."in.emissions_types",
resstock_amy2018_release_1_1_metadata."in.emissions_wood_values",
resstock_amy2018_release_1_1_metadata."in.federal_poverty_level", resstock_amy20
18_release_1_1_metadata."in.generation_and_emissions_assessment_region",

resstock_amy2018_release_1_1_metadata."in.geometry_attic_type", resstock_amy2018_release_1_1_metadata."in.geometry_building_horizontal_location_mf", resstock_amy2018_release_1_1_metadata."in.geometry_building_horizontal_location_sfa", resstock_amy2018_release_1_1_metadata."in.geometry_building_level_mf", resstock_amy2018_release_1_1_metadata."in.geometry_building_number_units_mf", resstock_amy2018_release_1_1_metadata."in.geometry_building_number_units_sfa", resstock_amy2018_release_1_1_metadata."in.geometry_building_type_acs", resstock_amy2018_release_1_1_metadata."in.geometry_building_type_height", resstock_amy2018_release_1_1_metadata."in.geometry_building_type_recs", resstock_amy2018_release_1_1_metadata."in.geometry_floor_area", resstock_amy2018_release_1_1_metadata."in.geometry_floor_area_bin", resstock_amy2018_release_1_1_metadata."in.geometry_foundation_type", resstock_amy2018_release_1_1_metadata."in.geometry_garage", resstock_amy2018_release_1_1_metadata."in.geometry_stories", resstock_amy2018_release_1_1_metadata."in.geometry_stories_low_rise", resstock_amy2018_release_1_1_metadata."in.geometry_story_bin", resstock_amy2018_release_1_1_metadata."in.geometry_wall_exterior_finish", resstock_amy2018_release_1_1_metadata."in.geometry_wall_type", resstock_amy2018_release_1_1_metadata."in.has_pv", resstock_amy2018_release_1_1_metadata."in.heating_fuel", resstock_amy2018_release_1_1_metadata."in.heating_setpoint", resstock_amy2018_release_1_1_metadata."in.heating_setpoint_has_offset", resstock_amy2018_release_1_1_metadata."in.heating_setpoint_offset_magnitude", resstock_amy2018_release_1_1_metadata."in.heating_setpoint_offset_period", resstock_amy2018_release_1_1_metadata."in.holiday_lighting", resstock_amy2018_release_1_1_metadata."in.hot_water_distribution", resstock_amy2018_release_1_1_metadata."in.hot_water_fixtures", resstock_amy2018_release_1_1_metadata."in.hvac_cooling_efficiency", resstock_amy2018_release_1_1_metadata."in.hvac_cooling_partial_space_conditioning", resstock_amy2018_release_1_1_metadata."in.hvac_cooling_type", resstock_amy2018_release_1_1_metadata."in.hvac_has_ducts", resstock_amy2018_release_1_1_metadata."in.hvac_has_shared_system", resstock_amy2018_release_1_1_metadata."in.hvac_has_zonal_electric_heating", resstock_amy2018_release_1_1_metadata."in.hvac_heating_efficiency", resstock_amy2018_release_1_1_metadata."in.hvac_heating_type", resstock_amy2018_release_1_1_metadata."in.hvac_heating_type_and_fuel", resstock_amy2018_release_1_1_metadata."in.hvac_secondary_heating_efficiency", resstock_amy2018_release_1_1_metadata."in.hvac_secondary_heating_type_and_fuel", resstock_amy2018_release_1_1_metadata."in.hvac_shared_efficiencies", resstock_amy2018_release_1_1_metadata."in.hvac_system_is_faulted", resstock_amy2018_release_1_1_metadata."in.hvac_system_single_speed_ac_airflow", resstock_amy2018_release_1_1_metadata."in.hvac_system_single_speed_ac_charge", resstock_amy2018_release_1_1_metadata."in.hvac_system_single_speed_ashp_airflow", resstock_amy2018_release_1_1_metadata."in.hvac_system_single_speed_ashp_charge", resstock_amy2018_release_1_1_metadata."in.income", resstock_amy2018_release_1_1_metadata."in.income_recs_2015", resstock_amy2018_release_1_1_metadata."in.income_recs_2020", resstock_amy2018_release_1_1_metadata."in.infiltration",

```

resstock_amy2018_release_1_1_metadata."in.insulation_ceiling",
resstock_amy2018_release_1_1_metadata."in.insulation_floor",
resstock_amy2018_release_1_1_metadata."in.insulation_foundation_wall",
resstock_amy2018_release_1_1_metadata."in.insulation_rim_joist",
resstock_amy2018_release_1_1_metadata."in.insulation_roof",
resstock_amy2018_release_1_1_metadata."in.insulation_slab",
resstock_amy2018_release_1_1_metadata."in.insulation_wall",
resstock_amy2018_release_1_1_metadata."in.interior_shading",
resstock_amy2018_release_1_1_metadata."in.iso_rto_region",
resstock_amy2018_release_1_1_metadata."in.lighting",
resstock_amy2018_release_1_1_metadata."in.lighting_interior_use",
resstock_amy2018_release_1_1_metadata."in.lighting_other_use",
resstock_amy2018_release_1_1_metadata."in.location_region",
resstock_amy2018_release_1_1_metadata."in.mechanical_ventilation",
resstock_amy2018_release_1_1_metadata."in.misc_extra_refrigerator",
resstock_amy2018_release_1_1_metadata."in.misc_freezer",
resstock_amy2018_release_1_1_metadata."in.misc_gas_fireplace",
resstock_amy2018_release_1_1_metadata."in.misc_gas_grill",
resstock_amy2018_release_1_1_metadata."in.misc_gas_lighting",
resstock_amy2018_release_1_1_metadata."in.misc_hot_tub_spa",
resstock_amy2018_release_1_1_metadata."in.misc_pool",
resstock_amy2018_release_1_1_metadata."in.misc_pool_heater",
resstock_amy2018_release_1_1_metadata."in.misc_pool_pump",
resstock_amy2018_release_1_1_metadata."in.misc_well_pump",
resstock_amy2018_release_1_1_metadata."in.natural_ventilation",
resstock_amy2018_release_1_1_metadata."in.neighbors",
resstock_amy2018_release_1_1_metadata."in.occupants",
resstock_amy2018_release_1_1_metadata."in.orientation",
resstock_amy2018_release_1_1_metadata."in.overhangs",
resstock_amy2018_release_1_1_metadata."in.plug_load_diversity",
resstock_amy2018_release_1_1_metadata."in.plug_loads",
resstock_amy2018_release_1_1_metadata."in.puma",
resstock_amy2018_release_1_1_metadata."in.puma_metro_status",
resstock_amy2018_release_1_1_metadata."in.pv_orientation",
resstock_amy2018_release_1_1_metadata."in.pv_system_size",
resstock_amy2018_release_1_1_metadata."in.radiant_barrier",
resstock_amy2018_release_1_1_metadata."in.range_spot_vent_hour",
resstock_amy2018_release_1_1_metadata."in.reeds_balancing_area",
resstock_amy2018_release_1_1_metadata."in.refrigerator",
resstock_amy2018_release_1_1_metadata."in.roof_material",
resstock_amy2018_release_1_1_metadata."in.schedules", resstock_amy2018_release_1
_1_metadata."in.simulation_control_run_period_begin_day_of_month", resstock_amy2
018_release_1_1_metadata."in.simulation_control_run_period_begin_month", resstoc
k_amy2018_release_1_1_metadata."in.simulation_control_run_period_calendar_year",
resstock_amy2018_release_1_1_metadata."in.simulation_control_run_period_end_day
_of_month", resstock_amy2018_release_1_1_metadata."in.simulation_control_run_peri
od_end_month",
resstock_amy2018_release_1_1_metadata."in.simulation_control_timestep",

```

```

resstock_amy2018_release_1_1_metadata."in.solar_hot_water",
resstock_amy2018_release_1_1_metadata."in.state",
resstock_amy2018_release_1_1_metadata."in.tenure",
resstock_amy2018_release_1_1_metadata."in.units_represented",
resstock_amy2018_release_1_1_metadata."in.usage_level",
resstock_amy2018_release_1_1_metadata."in.vacancy_status",
resstock_amy2018_release_1_1_metadata."in.vintage",
resstock_amy2018_release_1_1_metadata."in.vintage_acs",
resstock_amy2018_release_1_1_metadata."in.water_heater_efficiency",
resstock_amy2018_release_1_1_metadata."in.water_heater_fuel",
resstock_amy2018_release_1_1_metadata."in.water_heater_in_unit",
resstock_amy2018_release_1_1_metadata."in.weather_file_city",
resstock_amy2018_release_1_1_metadata."in.weather_file_latitude",
resstock_amy2018_release_1_1_metadata."in.weather_file_longitude",
resstock_amy2018_release_1_1_metadata."in.window_areas",
resstock_amy2018_release_1_1_metadata."in.windows",
resstock_amy2018_release_1_1_metadata."out.params.door_area_ft_2", resstock_amy2
018_release_1_1_metadata."out.params.duct_unconditioned_surface_area_ft_2",
resstock_amy2018_release_1_1_metadata."out.params.floor_area_attic_ft_2", ressto
ck_amy2018_release_1_1_metadata."out.params.floor_area_attic_insulation_increase
_ft_2_delta_r_value", resstock_amy2018_release_1_1_metadata."out.params.floor_ar
ea_conditioned_infiltration_reduction_ft_2_delta_ach_50",
resstock_amy2018_release_1_1_metadata."out.params.floor_area_foundation_ft_2",
resstock_amy2018_release_1_1_metadata."out.params.floor_area_lighting_ft_2", res
stock_amy2018_release_1_1_metadata."out.params.flow_rate_mechanical_ventilation_
cfm", resstock_amy2018_release_1_1_metadata."out.params.rim_joist_area_above_gra
de_exterior_ft_2",
resstock_amy2018_release_1_1_metadata."out.params.roof_area_ft_2", resstock_amy2
018_release_1_1_metadata."out.params.size_cooling_system_primary_k_btu_h", resst
ock_amy2018_release_1_1_metadata."out.params.size_heat_pump_backup_primary_k_btu
_h", resstock_amy2018_release_1_1_metadata."out.params.size_heating_system_prima
ry_k_btu_h", resstock_amy2018_release_1_1_metadata."out.params.size_heating_syst
em_secondary_k_btu_h",
resstock_amy2018_release_1_1_metadata."out.params.size_water_heater_gal", ressto
ck_amy2018_release_1_1_metadata."out.params.slab_perimeter_exposed_conditioned_f
t", resstock_amy2018_release_1_1_metadata."out.params.wall_area_above_grade_cond
itioned_ft_2", resstock_amy2018_release_1_1_metadata."out.params.wall_area_above
_grade_exterior_ft_2",
resstock_amy2018_release_1_1_metadata."out.params.wall_area_below_grade_ft_2",
resstock_amy2018_release_1_1_metadata."out.params.window_area_ft_2", resstock_am
y2018_release_1_1_metadata."out.electricity.total.lrmer_95_decarb_by_2035_15_202
5_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.fuel_oil.total.lrme
r_95_decarb_by_2035_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadat
a."out.natural_gas.total.lrmer_95_decarb_by_2035_15_2025_start.co2e_kg", resstoc
k_amy2018_release_1_1_metadata."out.propane.total.lrmer_95_decarb_by_2035_15_202
5_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.electricity.total.l
rmer_low_re_cost_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."
out.fuel_oil.total.lrmer_low_re_cost_15_2025_start.co2e_kg", resstock_amy2018_re

```

lease_1_1_metadata."out.natural_gas.total.lrmer_low_re_cost_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.propane.total.lrmer_low_re_cost_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.electricity.total.lrmer_low_re_cost_25_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.fuel_oil.total.lrmer_low_re_cost_25_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.natural_gas.total.lrmer_low_re_cost_25_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.propane.total.lrmer_low_re_cost_25_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.electricity.total.lrmer_mid_case_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.fuel_oil.total.lrmer_mid_case_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.natural_gas.total.lrmer_mid_case_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.propane.total.lrmer_mid_case_15_2025_start.co2e_kg", resstock_amy2018_release_1_1_metadata."out.electricity.ceiling_fan.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.ceiling_fan.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.clothes_dryer.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.clothes_dryer.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.clothes_washer.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.clothes_washer.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.cooling_fans_pumps.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.cooling_fans_pumps.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.cooling.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.cooling.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.dishwasher.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.dishwasher.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.freezer.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.freezer.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.heating_fans_pumps.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.heating_fans_pumps.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.heating_hp_bkup.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.heating_hp_bkup.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.heating.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.heating.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.hot_tub_heater.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.hot_tub_heater.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.hot_tub_pump.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.hot_tub_pump.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.hot_water.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.hot_water.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.lighting_exterior.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.lighting_exterior.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.lighting_garage.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.lighting_garage.energy_consumption_intensity", resstock_amy2018_release_

1_1_metadata."out.electricity.lighting_interior.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.lighting_interior.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.mech_vent.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.mech_vent.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.plug_loads.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.plug_loads.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.pool_heater.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.pool_heater.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.pool_pump.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.pool_pump.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.pv.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.pv.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.range_oven.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.range_oven.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.refrigerator.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.refrigerator.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.well_pump.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.well_pump.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.fuel_oil.heating_hp_bkup.energy_consumption", resstock_amy2018_release_1_1_metadata."out.fuel_oil.heating_hp_bkup.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.fuel_oil.heating.energy_consumption", resstock_amy2018_release_1_1_metadata."out.fuel_oil.heating.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.fuel_oil.hot_water.energy_consumption", resstock_amy2018_release_1_1_metadata."out.fuel_oil.hot_water.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.clothes_dryer.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.clothes_dryer.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.fireplace.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.fireplace.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.grill.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.grill.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.heating_hp_bkup.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.heating_hp_bkup.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.heating.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.heating.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.hot_tub_heater.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.hot_tub_heater.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.hot_water.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.hot_water.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.lighting.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.lighting.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.pool_heater.energy_consumption", resstock_amy20

18_release_1_1_metadata."out.natural_gas.pool_heater.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.range_oven.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.range_oven.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.clothes_dryer.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.clothes_dryer.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.heating_hp_bkup.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.heating_hp_bkup.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.heating.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.heating.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.hot_water.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.hot_water.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.range_oven.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.range_oven.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.site_energy.net.energy_consumption", resstock_amy2018_release_1_1_metadata."out.site_energy.net.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.site_energy.total.energy_consumption", resstock_amy2018_release_1_1_metadata."out.site_energy.total.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.net.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.net.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.electricity.total.energy_consumption", resstock_amy2018_release_1_1_metadata."out.electricity.total.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.fuel_oil.total.energy_consumption", resstock_amy2018_release_1_1_metadata."out.fuel_oil.total.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.natural_gas.total.energy_consumption", resstock_amy2018_release_1_1_metadata."out.natural_gas.total.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.propane.total.energy_consumption", resstock_amy2018_release_1_1_metadata."out.propane.total.energy_consumption_intensity", resstock_amy2018_release_1_1_metadata."out.hot_water.clothes_washer.gal", resstock_amy2018_release_1_1_metadata."out.hot_water.dishwasher.gal", resstock_amy2018_release_1_1_metadata."out.hot_water.distribution_waste.gal", resstock_amy2018_release_1_1_metadata."out.hot_water.fixtures.gal", resstock_amy2018_release_1_1_metadata."out.load.cooling.energy_delivered.kbtu", resstock_amy2018_release_1_1_metadata."out.load.heating.energy_delivered.kbtu", resstock_amy2018_release_1_1_metadata."out.load.hot_water.energy_delivered.kbtu", resstock_amy2018_release_1_1_metadata."out.electricity.summer.peak.kw", resstock_amy2018_release_1_1_metadata."out.electricity.winter.peak.kw", resstock_amy2018_release_1_1_metadata."out.load.cooling.peak.kbtu_hr", resstock_amy2018_release_1_1_metadata."out.load.heating.peak.kbtu_hr", resstock_amy2018_release_1_1_metadata."out.unmet_hours.cooling.hour", resstock_amy2018_release_1_1_metadata."out.unmet_hours.heating.hour", resstock_amy2018_release_1_1_metadata.upgrade, resstock_amy2018_release_1_1_metadata.metadata_index, resstock_amy2018_release_1_1_metadata.bldg_id

```
FROM resstock_amy2018_release_1_1_metadata
WHERE CAST(resstock_amy2018_release_1_1_metadata.upgrade AS VARCHAR) = :param_1
Timeseries table: resstock_amy2018_release_1_1_by_state
Building ID col : bldg_id
```

Step 1 COMPLETE

```
[5]: # =====
# Step 2: Column Name Constants (imported from cmu_tare_model.constants)
# =====
# BSQ column names and Allegheny County reference values are imported from
# cmu_tare_model.constants (see Step 0 imports). Weights are handled by BSQ
# internally via SUM(enduse * baseline.weight) in generated SQL.

print(f"""
    Step 2: Column constants (imported from constants.py)

    BLDG_ID_COL      : {BLDG_ID_COL}
    ELEC_TOTAL_COL   : {ELEC_TOTAL_COL}
    BSQ_ELEC_COL     : {BSQ_ELEC_COL}
    TEST_FIPS        : {TEST_FIPS}
    """)
```

Step 2: Column constants (imported from constants.py)

```
BLDG_ID_COL      : bldg_id
ELEC_TOTAL_COL   : out.electricity.total.energy_consumption
BSQ_ELEC_COL     : electricity.total.energy_consumption
TEST_FIPS        : 42003
```

Step 3: County Geography Mapping — FIPS / GISJOIN to Shapefile

ResStock EUSS uses either FIPS codes or GISJOIN identifiers for county-level geography. We need to confirm which identifier is present and map it to standard Census TIGER/Line shapefiles for choropleth visualization.

GISJOIN format: G + state FIPS (2 digits, zero-padded) + 0 + county FIPS (3 digits, zero-padded). Example: Allegheny County PA = G4200030.

Standard FIPS format: 5-digit string. Example: Allegheny County PA = 42003.

Goal of this step: Build a lookup table `county_geo_df` that maps ResStock's county identifier → FIPS → county name → state, so all downstream results can be joined to shapefiles.

Test case: Confirm that Allegheny County, PA appears with FIPS 42003.

```

[6]: # County shapefile path (separate from the state-level SHAPEFILE_PATH)
COUNTY_SHAPEFILE_PATH: str = os.path.join(
    PROJECT_ROOT, "cmu_tare_model", "data", "electricity_ng_price_ratio",
    "tl_2025_us_county", "tl_2025_us_county.shp"
)

# TIGER/Line county shapefiles: GEOID = 5-digit FIPS, NAME = county name,
# STATEFP = 2-digit state FIPS (no state abbreviation in county shapefiles).
TIGER_FIPS_COL: Final[str] = "GEOID"
TIGER_NAME_COL: Final[str] = "NAME"
TIGER_STATEFP_COL: Final[str] = "STATEFP"

# Load the county shapefile
gdf_counties_raw: gpd.GeoDataFrame = gpd.read_file(COUNTY_SHAPEFILE_PATH)

# Validate expected columns exist
_expected_cols = {TIGER_FIPS_COL, TIGER_NAME_COL, TIGER_STATEFP_COL}
_missing = _expected_cols - set(gdf_counties_raw.columns)
if _missing:
    raise KeyError(
        f"TIGER county shapefile missing expected columns: {_missing}\n"
        f" Available: {sorted(gdf_counties_raw.columns.tolist())}\n"
        f" Path: {COUNTY_SHAPEFILE_PATH}"
    )

# Build the plain lookup DataFrame
county_geo_df: pd.DataFrame = gdf_counties_raw[
    [TIGER_FIPS_COL, TIGER_NAME_COL, TIGER_STATEFP_COL]
].rename(
    columns={
        TIGER_FIPS_COL: "fips_5digit",
        TIGER_NAME_COL: "county_name",
        TIGER_STATEFP_COL: "state_fips",
    }
).copy()
county_geo_df["fips_5digit"] = county_geo_df["fips_5digit"].astype(str).
    ↪zfill(5)

# GeoDataFrame with renamed columns for downstream merges
gdf_counties: gpd.GeoDataFrame = gdf_counties_raw.rename(
    columns={
        TIGER_FIPS_COL: "fips_5digit",
        TIGER_NAME_COL: "county_name",
        TIGER_STATEFP_COL: "state_fips",
    }
)[["fips_5digit", "county_name", "state_fips", "geometry"]]

```

```

gdf_counties["fips_5digit"] = gdf_counties["fips_5digit"].astype(str).str.
    ↪zfill(5)

# Validation: Allegheny County, PA (FIPS 42003) must be present
test_row = county_geo_df[county_geo_df["fips_5digit"] == TEST_FIPS]
if test_row.empty:
    raise ValueError(
        f"Test county FIPS {TEST_FIPS} (Allegheny, PA) not found in shapefile.
    ↪\n"
        f" This blocks the Step 7 validation case. Check COUNTY_SHAPEFILE_PATH.
    ↪"
    )
print(f" county_geo_df: {len(county_geo_df):,d} counties")
print(f" Test county: {test_row.iloc[0].to_dict()}")
print(f" gdf_counties CRS: {gdf_counties.crs}")

```

```

county_geo_df: 3,235 counties
Test county: {'fips_5digit': '42003', 'county_name': 'Allegheny',
'state_fips': '42'}
gdf_counties CRS: EPSG:4269

```

Step 4: Extract Adopter Building IDs from TARE Results

The TARE model assigns each building (`bldg_id`) to an adoption tier: - **Tier 1** — Private NPV positive (saves money without incentives) - **Tier 2** — Private NPV positive only with IRA rebates - **Tier 3** — Requires public benefits (social cost of carbon) to be justified - **Tier 4** — No adoption pathway under current conditions

For the **economically-constrained scenario**, adopters = Tier 1 + Tier 2. For the **100% adoption counterfactual**, adopters = all filtered building IDs.

This step extracts both sets of building IDs, organized by county (FIPS), for use in the timeseries queries in Steps 5–6.

Output structure:

```

adopter_ids_by_county = {
    "42003": {                                     # FIPS for Allegheny County, PA
        "tier1": [101, 204, ...],
        "tier2": [305, 412, ...],
        "constrained": [101, 204, 305, 412, ...], # tier1 + tier2
        "all_filtered": [101, 204, ..., 99999],    # 100% adoption
    }
}

```

```

[7]: # ===== Configuration =====
primary_mp: int = selected_mps[0]
DISCOUNT_RATE_KEY: str = "fixed_base"
RCM_MODEL_KEY: str = "inmap"

```

```

# ===== 1. Access the TARE DataFrame =====
print(f"Primary MP: {primary_mp}")
print(f"Discount rate key: {DISCOUNT_RATE_KEY}")
print(f"RCM model key: {RCM_MODEL_KEY}")

df_tare_nested = DATAFRAMES_BY_MP[primary_mp][DISCOUNT_RATE_KEY]

# Handle nesting: DATAFRAMES_BY_MP[mp][dr_key] may be {rcm_model: DataFrame}
if isinstance(df_tare_nested, dict):
    if RCM_MODEL_KEY not in df_tare_nested:
        raise KeyError(
            f"RCM model '{RCM_MODEL_KEY}' not found. "
            f"Available: {list(df_tare_nested.keys())}"
        )
    df_tare: pd.DataFrame = df_tare_nested[RCM_MODEL_KEY]
    print(f" Accessed_
↳DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']['{RCM_MODEL_KEY}']")
else:
    df_tare = df_tare_nested
    print(f" Accessed DATAFRAMES_BY_MP[{primary_mp}]['{DISCOUNT_RATE_KEY}']_
↳(direct DataFrame)")

print(f" Shape: {df_tare.shape}")
print(f" Index name: {df_tare.index.name}")

# ===== 2. Find the adoption column =====
# Try each cost scenario in priority order until one matches.
adoption_col: str | None = None
for cs in REMDB_COST_SCENARIO_KEYS:
    try:
        adoption_col = find_adoption_column(df_tare, primary_mp, cs)
        print(f"\n Adoption column found (cost_scenario='{cs}'):\n _
↳{adoption_col}")
        break
    except KeyError:
        continue

if adoption_col is None:
    # Final attempt - raise with full diagnostics from the first cost scenario
    adoption_col = find_adoption_column(df_tare, primary_mp,
↳REMDB_COST_SCENARIO_KEYS[0])

# Print tier distribution
tier_counts = df_tare[adoption_col].value_counts()
print(f"\n===== Tier distribution (MP={primary_mp}) =====")
for tier_val, count in tier_counts.items():

```

```

    print(f"   {tier_val:<45s}   {count:>7,d}")
print(f"   {'TOTAL':<45s}   {tier_counts.sum():>7,d}")

# ===== 3. Extract adopter IDs by county =====
adopter_ids_by_county: dict[str, dict[str, list[int]]] = extract_adopter_ids(
    df_tare, adoption_col
)

n_counties = len(adopter_ids_by_county)
total_constrained = sum(len(v["constrained"]) for v in adopter_ids_by_county.
    ↪values())
total_all = sum(len(v["all_filtered"]) for v in adopter_ids_by_county.values())
print(f"\n===== Adopter summary =====")
print(f"   Counties           : {n_counties:,d}")
print(f"   Constrained (T1+T2): {total_constrained:,d}")
print(f"   All filtered        : {total_all:,d}")

# ===== 4. Test case: Allegheny County, PA (FIPS 42003) =====
if TEST_FIPS in adopter_ids_by_county:
    ac = adopter_ids_by_county[TEST_FIPS]
    print(f"\n===== Allegheny County (FIPS {TEST_FIPS}) =====")
    print(f"   Tier 1           : {len(ac['tier1']):,d}")
    print(f"   Tier 2           : {len(ac['tier2']):,d}")
    print(f"   Constrained      : {len(ac['constrained']):,d}")
    print(f"   All filtered     : {len(ac['all_filtered']):,d}")
else:
    print(f"\n FIPS {TEST_FIPS} (Allegheny County, PA) not found in results.")
    sample_fips = list(adopter_ids_by_county.keys())[:10]
    print(f"   Available FIPS (first 10): {sample_fips}")

print("\n Step 4 COMPLETE")

```

Primary MP: 3

Discount rate key: fixed_base

RCM model key: inmap

Accessed DATAFRAMES_BY_MP[3]['fixed_base']['inmap']

Shape: (331531, 271)

Index name: bldg_id

Adoption column found (cost_scenario='v3'):

iraRef_mp3_heating_adoption_central_inmap_acs_v3_fixed_base

===== Tier distribution (MP3) =====

Tier 4: Averse	151,340
Tier 3: Subsidy-Dependent Feasibility	106,932
Tier 1: Feasible	16,650
Tier 2: Feasible vs. Alternative	16,453
N/A: Invalid Baseline Fuel/Tech	183

TOTAL 291,558

===== Adopter summary =====

Counties : 3,098
Constrained (T1+T2): 33,103
All filtered : 331,531

===== Allegheny County (FIPS 42003) =====

Tier 1 : 64
Tier 2 : 29
Constrained : 93
All filtered : 1,610

Step 4 COMPLETE

Step 5: Baseline Timeseries — Allegheny County (BSQ)

Queries baseline (upgrade=0) hourly electricity consumption for all buildings in Allegheny County via `BuildStockQuery.agg.aggregate_timeseries()`.

BSQ generates SQL that: - Joins timeseries $\leftarrow$ metadata on `bldg_id` - Aggregates 15-min intervals to hourly via `date_trunc('hour', ...)` - Applies `SUM(enduse  $\times$  weight)` — values are **weight-applied** - Returns per-building data when `group_by=['bldg_id']`

Note: `split_enduses=False` required (True triggers Pydantic bug in BSQ batch path).

```
[8]: # =====  
# STEP 5: Baseline Timeseries - Allegheny County (BSQ)  
# =====  
allegheny_bldg_ids: list[int] = adopter_ids_by_county[TEST_FIPS] ["all_filtered"]  
print(f" Allegheny bldg_ids (all_filtered): {len(allegheny_bldg_ids):,d}")  
  
# ===== Query baseline timeseries via BSQ =====  
print(f"\n Querying baseline timeseries (upgrade=0, {len(allegheny_bldg_ids)}  
      ↪ buildings)...")  
t_start = time.perf_counter()  
  
ts_query_baseline = TSQuery(  
    enduses=[ELEC_TOTAL_COL],  
    restrict=[('bldg_id', allegheny_bldg_ids)],  
    upgrade_id='0',  
    timestamp_grouping_func='hour',  
    group_by=[BLDG_ID_COL],  
    split_enduses=False,  
)  
  
df_ts_baseline_allegheny: pd.DataFrame = my_run.agg.aggregate_timeseries(  
    params=ts_query_baseline
```

```

)
query_time_s: float = time.perf_counter() - t_start

# ===== Rename and downcast =====
df_ts_baseline_alleggheny = df_ts_baseline_alleggheny.rename(
    columns={BSQ_ELEC_COL: 'baseline_kwh'}
)
# float32 is sufficient for kWh values and halves memory at national scale
df_ts_baseline_alleggheny['baseline_kwh'] =
↳df_ts_baseline_alleggheny['baseline_kwh'].astype(np.float32)

# Deterministic hour index: sort by (bldg_id, timestamp) before assigning
↳ordinals
df_ts_baseline_alleggheny = df_ts_baseline_alleggheny.sort_values(
    [BLDG_ID_COL, TIMESTAMP_COL]
).reset_index(drop=True)
df_ts_baseline_alleggheny['hour'] = (
    df_ts_baseline_alleggheny.groupby(BLDG_ID_COL).cumcount() + 1
)

# ===== Summary =====
n_bldgs: int = df_ts_baseline_alleggheny[BLDG_ID_COL].nunique()
n_hours_per_bldg = df_ts_baseline_alleggheny.groupby(BLDG_ID_COL).size()
weight_val: float = df_ts_baseline_alleggheny['units_count'].iloc[0]

print(f"\n===== df_ts_baseline_alleggheny summary =====")
print(f"  Rows           : {len(df_ts_baseline_alleggheny):,d}")
print(f"  Buildings        : {n_bldgs:,d}")
print(f"  Hours/bldg       : {n_hours_per_bldg.min():,d} - {n_hours_per_bldg.
↳max():,d}")
print(f"  kWh range (wtd) : {df_ts_baseline_alleggheny['baseline_kwh'].min():.
↳3f}")
print(f"                  : {df_ts_baseline_alleggheny['baseline_kwh'].max():.3f}")
print(f"  BSQ weight       : {weight_val:.6f}")
print(f"  Query time (s)   : {query_time_s:.2f}")

assert n_hours_per_bldg.min() == 8760, f"Min hours/bldg = {n_hours_per_bldg.
↳min()}, expected 8760"
assert n_hours_per_bldg.max() == 8760, f"Max hours/bldg = {n_hours_per_bldg.
↳max()}, expected 8760"
print(f"\n Step 5 PASSED - {n_bldgs} buildings × 8,760 hours")
display(df_ts_baseline_alleggheny.head())

```

Allegheny bldg_ids (all_filtered): 1,610

Querying baseline timeseries (upgrade=0, 1610 buildings)...

WARNING:buildstock_query.query_core:Column bldg_id found in multiple tables

```
['resstock_amy2018_release_1_1_by_state', 'baseline', 'upgrade']. Using
resstock_amy2018_release_1_1_by_state
INFO:buildstock_query.aggregate_query:Restricting query to Upgrade 0.
WARNING:buildstock_query.query_core:Column bldg_id found in multiple tables
['resstock_amy2018_release_1_1_by_state', 'baseline', 'upgrade']. Using
resstock_amy2018_release_1_1_by_state
INFO:pyathena.pandas.result_set:Reading 1184663251 bytes from S3 in full mode
using c engine
```

```
===== df_ts_baseline_alleggheny summary =====
```

```
Rows           : 14,103,600
Buildings      : 1,610
Hours/bldg     : 8,760 - 8,760
kWh range (wtd) : 31.961 to 10401.222
BSQ weight     : 242.131013
Query time (s) : 174.54
```

Step 5 PASSED - 1610 buildings × 8,760 hours

	bldg_id	timestamp	sample_count	units_count	rows_per_sample	\
0	491	2018-01-01 00:00:00	1	242.131013		4
1	491	2018-01-01 01:00:00	1	242.131013		4
2	491	2018-01-01 02:00:00	1	242.131013		4
3	491	2018-01-01 03:00:00	1	242.131013		4
4	491	2018-01-01 04:00:00	1	242.131013		4

	baseline_kwh	hour
0	106.537643	1
1	119.612717	2
2	86.440773	3
3	93.220436	4
4	96.126015	5

Step 6: Upgrade Timeseries (MP3 or MP4) — Allegheny County (BSQ)

Queries post-retrofit timeseries for the same county and building IDs. Same BSQ approach as Step 5 with `upgrade_id=str(primary_mp)`.

Only buildings where `applicability == True` have valid upgrade data. Buildings in `all_filtered` with no upgrade data will fall back to baseline in Step 7.

```
[9]: # =====
# STEP 6: Upgrade Timeseries - Allegheny County (BSQ)
# =====
print(f"\n Querying upgrade timeseries (upgrade={primary_mp})...")
t_start = time.perf_counter()
```

```

ts_query_upgrade = TSQuery(
    enduses=[ELEC_TOTAL_COL],
    restrict=[('bldg_id', allegheny_bldg_ids)],
    upgrade_id=str(primary_mp),
    timestamp_grouping_func='hour',
    group_by=[BLDG_ID_COL],
    split_enduses=False,
)

df_ts_upgrade_allegheny: pd.DataFrame = my_run.agg.aggregate_timeseries(
    params=ts_query_upgrade
)

upgrade_query_time_s: float = time.perf_counter() - t_start

# ===== Rename and downcast =====
df_ts_upgrade_allegheny = df_ts_upgrade_allegheny.rename(
    columns={BSQ_ELEC_COL: 'retrofit_kwh'}
)

# float32 is sufficient for kWh values and halves memory at national scale
df_ts_upgrade_allegheny['retrofit_kwh'] =
    df_ts_upgrade_allegheny['retrofit_kwh'].astype(np.float32)

# Deterministic hour index
df_ts_upgrade_allegheny = df_ts_upgrade_allegheny.sort_values(
    [BLDG_ID_COL, TIMESTAMP_COL]
).reset_index(drop=True)
df_ts_upgrade_allegheny['hour'] = (
    df_ts_upgrade_allegheny.groupby(BLDG_ID_COL).cumcount() + 1
)

# ===== Schema parity check =====
baseline_bldgs: set[int] = set(df_ts_baseline_allegheny[BLDG_ID_COL].unique())
upgrade_bldgs: set[int] = set(df_ts_upgrade_allegheny[BLDG_ID_COL].unique())
only_in_baseline: set[int] = baseline_bldgs - upgrade_bldgs
only_in_upgrade: set[int] = upgrade_bldgs - baseline_bldgs

n_hours_up = df_ts_upgrade_allegheny.groupby(BLDG_ID_COL).size()

print(f"\n===== df_ts_upgrade_allegheny summary =====")
print(f" Rows : {len(df_ts_upgrade_allegheny):,d}")
print(f" Buildings : {len(upgrade_bldgs):,d}")
print(f" Hours/bldg : {n_hours_up.min()} - {n_hours_up.max()}")
print(f" Baseline bldgs : {len(baseline_bldgs):,d}")
print(f" Only in baseline: {len(only_in_baseline):,d}")
print(f" Only in upgrade : {len(only_in_upgrade):,d}")
print(f" kWh range (wtd) : {df_ts_upgrade_allegheny['retrofit_kwh'].min():.3f}"
      f" to {df_ts_upgrade_allegheny['retrofit_kwh'].max():.3f}")

```

```

print(f"  Query time (s)   : {upgrade_query_time_s:.2f}")

if only_in_baseline:
    print(f"\n  Note: {len(only_in_baseline):,d} buildings have no upgrade data,
    ↳ will use baseline in Step 7.")
if only_in_upgrade:
    raise ValueError(f"{len(only_in_upgrade)} bldg_ids in upgrade but not
    ↳baseline. Investigate.")

print(f"\n  Step 6 PASSED")
display(df_ts_upgrade_allegheny.head())

```

Querying upgrade timeseries (upgrade=3)...

```

WARNING:buildstock_query.query_core:Column bldg_id found in multiple tables
['resstock_amy2018_release_1_1_by_state', 'baseline', 'upgrade']. Using
resstock_amy2018_release_1_1_by_state
INFO:buildstock_query.aggregate_query:Restricting query to Upgrade 3.
WARNING:buildstock_query.query_core:Column bldg_id found in multiple tables
['resstock_amy2018_release_1_1_by_state', 'baseline', 'upgrade']. Using
resstock_amy2018_release_1_1_by_state
INFO:pyathena.pandas.result_set:Reading 1183620662 bytes from S3 in full mode
using c engine

```

===== df_ts_upgrade_allegheny summary =====

```

Rows           : 14,103,600
Buildings      : 1,610
Hours/bldg     : 8760 - 8760
Baseline bldgs : 1,610
Only in baseline: 0
Only in upgrade : 0
kWh range (wtd) : 32.930 to 25086.951
Query time (s)  : 175.95

```

Step 6 PASSED

	bldg_id	timestamp	sample_count	units_count	rows_per_sample	\
0	491	2018-01-01 00:00:00	1	242.131013	4	
1	491	2018-01-01 01:00:00	1	242.131013	4	
2	491	2018-01-01 02:00:00	1	242.131013	4	
3	491	2018-01-01 03:00:00	1	242.131013	4	
4	491	2018-01-01 04:00:00	1	242.131013	4	

	retrofit_kwh	hour
0	1036.078613	1
1	1313.802856	2
2	306.779999	3

3	515.254822	4
4	658.354248	5

Step 7: Compute Scenario Demand Profile — Allegheny County (Test Case)

Applies the adopter/non-adopter mask to produce a scenario hourly demand profile.

Logic (per building, per hour): - If `bldg_id` is in `adopter_ids` → use `retrofit_kwh` (post-HP electricity) - Otherwise → use `baseline_kwh` (existing equipment electricity)

Two profiles produced: 1. **100% adoption:** all filtered buildings adopt → all use `retrofit_kwh`
 2. **Constrained adoption:** Tier 1 + Tier 2 adopt; Tier 3 + Tier 4 use baseline

Peak load change = $\max(\text{scenario_profile_mw}) - \max(\text{baseline_profile_mw})$

Units note: BSQ returns weight-applied kWh (= raw kWh × 242.131013). Dividing by 1000 converts to MW. No separate sampling weight multiplication needed.

```
[10]: # --- Compute profiles for Allegheny County ---
print(" Computing 100% adoption profile...")
df_profile_100pct, peak_100pct = compute_county_scenario_profile(
    df_ts_baseline_alleggheny,
    df_ts_upgrade_alleggheny,
    adopter_bldg_ids=adopter_ids_by_county[TEST_FIPS]["all_filtered"],
)

print(" Computing constrained adoption profile (Tier 1+2)...")
df_profile_constrained, peak_constrained = compute_county_scenario_profile(
    df_ts_baseline_alleggheny,
    df_ts_upgrade_alleggheny,
    adopter_bldg_ids=adopter_ids_by_county[TEST_FIPS]["constrained"],
)

peak_results_alleggheny: dict[str, dict[str, Any]] = {
    "100pct": peak_100pct,
    "constrained": peak_constrained,
}

print(f"\n Allegheny peak results (MP{primary_mp})")
for scenario, p in peak_results_alleggheny.items():
    print(f"\n  [{scenario}]")
    print(f"    adopters      : {p['n_adopters']:,d} / {p['n_total_buildings']:,d}")
    print(f"    baseline peak : {p['baseline_peak_mw']:.2f} MW @ hour_{p['peak_hour_baseline']}")
    print(f"    scenario peak : {p['scenario_peak_mw']:.2f} MW @ hour_{p['peak_hour_scenario']}")
    print(f"    delta         : {p['delta_mw']:+.2f} MW")
```

```

print(f"\n Validation: len(df_profile_100pct) = {len(df_profile_100pct)}")
print(f" Validation: len(df_profile_constrained) = {len(df_profile_constrained)}")
assert len(df_profile_100pct) == 8760, "100pct profile not 8760 rows!"
assert len(df_profile_constrained) == 8760, "constrained profile not 8760 rows!"
print("\n Step 7 PASSED")

```

Computing 100% adoption profile...

Computing constrained adoption profile (Tier 1+2)...

Allegheny peak results (MP3)

[100pct]

```

adopters      : 1,610 / 1,610
baseline peak : 862.51 MW @ hour 4433
scenario peak  : 6629.87 MW @ hour 152
delta         : +5767.36 MW

```

[constrained]

```

adopters      : 93 / 1,610
baseline peak : 862.51 MW @ hour 4433
scenario peak  : 885.63 MW @ hour 116
delta         : +23.12 MW

```

Validation: len(df_profile_100pct) = 8760

Validation: len(df_profile_constrained) = 8760

Step 7 PASSED

Step 8: Validate — Compare Aggregated Profile Against EUSS Peak Load Columns

EUSS annual/metadata files include individual household peak load values (kW). These provide a within-dataset sanity check before comparing to external benchmarks.

Validation approach: - Sum the EUSS individual peak load values for Allegheny County buildings → county peak (naive sum) - Compare to the profile-derived peak from Step 7 (baseline, 100% adoption) - These will not be identical (naive sum coincident peak), but should be same order of magnitude

External benchmark (after internal check passes): - Compare PA statewide peak load change to Maxim et al. (2024) 2018 vs. 2050 scenarios - Reference OEDI URL from notebook stub: https://data.openei.org/s3_viewer?bucket=oedi-data-lake&prefix=nrel-pds-building-stock%2Fend-use

Flag: If profile-derived peak differs from EUSS column sum by more than 20%, investigate before scaling to national loop.

```

[11]: # Step 8 stub - see post_tare_review_cleanup_plan_v2.md for spec.
      # Validate aggregated profile against EUSS annual peak load columns.

```

```
raise NotImplementedError("Step 8 - validate against EUSS metadata peak load_
↳columns")
```

```
-----
NotImplementedError                                Traceback (most recent call last)
Cell In[11], line 3
      1 # Step 8 stub - see post_tare_review_cleanup_plan_v2.md for spec.
      2 # Validate aggregated profile against EUSS annual peak load columns.
----> 3 raise NotImplementedError("Step 8 - validate against EUSS metadata peak
↳load columns")
```

```
NotImplementedError: Step 8 - validate against EUSS metadata peak load columns
```

Step 9: National Loop — County-Level Peak Load Table

Run Step 7 validation on Allegheny County BEFORE running this step.

Scales the Allegheny County pipeline across all counties present in adopter_ids_by_county.

Design decisions to resolve before running: 1. **Query batching:** Query all buildings for a county in one Athena call, or batch by state first (reduces number of Athena connections)? 2. **Aggregation location:** Push the hourly sum to Athena SQL (cheaper, faster) or pull building-level data and aggregate in pandas (more flexible for adopter mask)? 3. **Checkpoint saving:** Save intermediate results per state to disk in case the loop fails mid-run (strongly recommended for a national run).

Expected output: df_peak_results_national with one row per county: [fips, county_name, state, n_adopters_constrained, n_all_filtered, baseline_peak_mw, scenario_100pct_peak_mw, scenario_constrained_peak_mw, delta_100pct_mw, delta_constrained_mw, peak_hour_100pct, peak_hour_constrained]

```
[ ]: # Step 9 stub - see post_tare_review_cleanup_plan_v2.md for spec.
      # Scale county-level pipeline nationally with per-state checkpointing.
```

```
def run_national_peak_load_loop(
    bsq: "BuildStockQuery",
    adopter_ids_by_county: dict[str, dict[str, list[int]]],
    county_geo_df: pd.DataFrame,
    primary_mp: int,
    project_root: str,
) -> pd.DataFrame:
    """Scale Steps 5-7 across all counties with per-state checkpointing.

    See post_tare_review_cleanup_plan_v2.md Step 9 spec for design decisions
    (query batching strategy, aggregation location, checkpoint format).

    Args:
```

```

        bsq: Initialized BuildStockQuery object.
        adopter_ids_by_county: Per-county adopter IDs from Step 4.
        county_geo_df: County FIPS -> name -> state lookup from Step 3.
        primary_mp: Measure package number (e.g. 3 or 4).
        project_root: Project root for checkpoint file paths.

    Returns:
        DataFrame with one row per county and peak load results.
    """
    raise NotImplementedError("Step 9 - implement after performance profiling, ↵
↵(Task 5)")

```

Step 10: Export Results for Paper Figures

Exports the national county-level peak load table for use in: - **Figure XX (paper Section 3.6):** County choropleth of peak load change under 100% adoption and economically-constrained adoption, by technology scenario (MP3/MP4) - **Allegheny County case study panel:** Bar chart comparing baseline vs scenario peak across all technology scenarios

Files exported: - `peak_load_results_MP{mp}_national.csv` — full national county table - `peak_load_results_MP{mp}_allegheny.csv` — Allegheny County only (for case study)

```

[ ]: # Step 10 stub - see post_tare_review_cleanup_plan_v2.md for spec.
      # Export national peak load results as CSV for paper figures.
      raise NotImplementedError("Step 10 - implement after Step 9 completes")

```